

Estudio de la convergencia e introducción a JAGS como herramienta (básica) de simulación MCMC

Introducción a la inferencia bayesiana basada en simulación

Miguel Ángel Martínez Beneito
miguel.a.martinez@uv.es



Sección 1

Análisis de la convergencia en MCMC

Análisis de la convergencia en MCMC

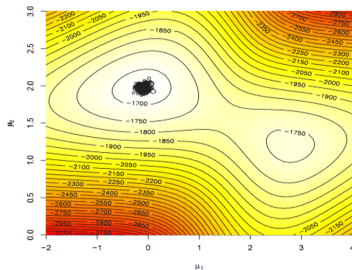
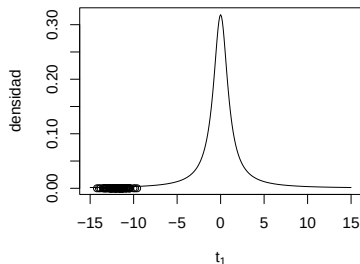
- El que **MCMC** simule **cadena de Markov** en lugar de muestras independientes hace que, en la práctica, tengamos que prestar particular **atención** a algunas **cuestiones**.
- El estudio de estos problemas se conoce como “**exploración de la convergencia**” y comprende el estudio de, al menos, las siguientes cuestiones:
 - ¿Cómo podemos tener “garantías” de que la simulación ha explorado **completamente el conjunto de valores** que debería visitar de la **distribución posterior**?
 - ¿Cómo evitamos el efecto de los **valores iniciales** arbitrarios empleados en nuestra simulación (cadena)?
 - ¿Qué **tamaño muestral** necesitamos en nuestra simulación?
 - ¿Cómo hacer los valores de nuestra muestra (**más**) “**independientes**”?
 - En un **algoritmo MH** de caminata aleatoria ¿qué **variabilidad** resulta **adecuada** para la función de propuesta $q(\cdot | \cdot)$?

Exploración completa de la posterior

- Pueden existir situaciones patológicas en las que la distribución a posteriori es **difícil de muestrear**, concretamente:

- Distribuciones a posteriori con **colas muy pesadas**.
- Distribuciones a posteriori **multimodales**.

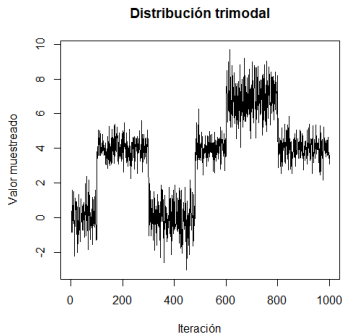
Muestra a posteriori de distribución t_1



- En estos casos las **muestras** obtenidas pueden **no** ser **representativas** de la distribución posterior que queremos muestrear.

- Para “asegurarnos” de que hemos explorado todo el rango de valores de la distribución a posteriori correremos **varias simulaciones** (de 3 a 5), cadenas, desde puntos iniciales **distantes**.
- Si todas las **cadenas convergen** a cierto valor tendremos “ciertas garantías” de que hemos recorrido **todo el rango** de valores razonables de la posterior.

El **comportamiento** de las **cadenas** hasta la convergencia (zonas en las que puedan quedar atrapadas) nos informa de posibles **patologías** en la distribución a posteriori.

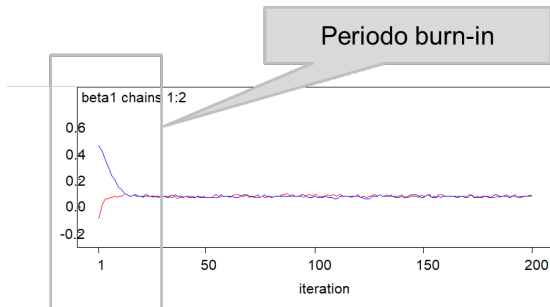


- La **visualización** de las cadenas generadas (`MCMCvis::MCMCtrace`) es una **herramienta** fundamental para valorar su **convergencia**.

Elección periodo de calentamiento (burn-in)

- La **parte inicial** de las cadenas puede depender de sus valores iniciales (**arbitrarios**).
- ¿Cómo **evitar el efecto** de las iteraciones iniciales sobre las cadenas? **Eliminando** su parte inicial (*Periodo de burn-in* o de *calentamiento* de la(s) cadena(s)).
- Para la determinación del número de simulaciones a descartar podemos **lanzar**, nuevamente, **varias cadenas** distintas, comenzando en **lugares diferentes**.

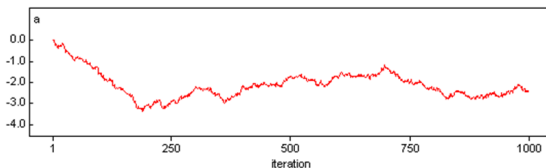
La **inspección visual** de las cadenas nos ayudará a determinar el número de **iteraciones** a descartar.



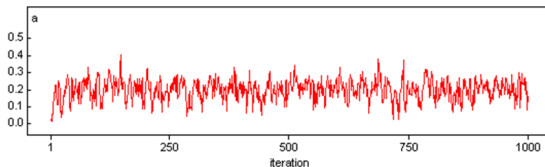
- En la práctica se suele tomar un **periodo de burn-in** supuestamente **largo** y se valora, para cada variable, si ese periodo es suficiente con el estadístico de **Gelman-Rubin**.
- El estadístico calcula, para las 50 iteraciones posteriores al burn-in, el **cociente** de:
 - **Amplitud** del intervalo de credibilidad al 80 % de la **muestra combinada** de todas las cadenas.
 - **Promedio de amplitudes** del intervalo de credibilidad al 80 % de cada una de las cadenas (por separado).
- Este estadístico se conoce como **R de Gelman-Rubin**. Si es **próximo a 1** el periodo burn-in elegido es adecuado.
- Suele ser habitual exigir que el estadístico de Gelman-Rubin sea **inferior a 1.1** para todos los parámetros del modelo.
- En general el **periodo de burn-in** debería estar en torno al **10 % de la longitud** total de las cadenas simuladas.

Número de iteraciones a simular

- Cuanto **mayor** sea el tamaño muestral generado **más precisa** será la inferencia.
- La **autocorrelación**/dependencia de las cadenas juega **en contra** de la inferencia que vamos a hacer.
- **No es lo mismo** una cadena del tipo:



que:



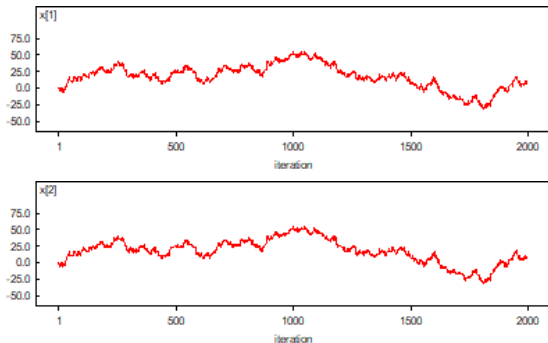
- El *número efectivo de simulaciones* ($n.eff$) es la herramienta **más habitual** para valorar la bondad del tamaño muestral simulado.
- Este estadístico tiene en cuenta, para cada variable, tanto la **longitud** de la cadena como su **autocorrelación**. Su interpretación es la del **tamaño muestral** de simulaciones **independientes** (descontando la autocorrelación de cada cadena) a la que equivale la cadena de Markov de la variable correspondiente.
- Si sólo queremos obtener un **estimador puntual** (media a posteriori) de un parámetro, puede ser suficiente un valor de su **$n.eff$ de 100**. Otros valores más difíciles de estimar, por ejemplo **percentiles** al 2.5 o al 97.5 %, pueden requerir valores algo **más altos**.
- El que **todas** las variables simuladas tengan un **$n.eff$ superior a 100** suele emplearse como criterio de calidad en simulación MCMC.
- Tanto el **estadístico** R de Gelman-Rubin como $n.eff$ serán **calculados automáticamente** por R según veremos en breve.

Disminución de la autocorrelación

- Querremos que nuestras cadenas presenten **autocorrelación baja** para que los **n.eff aumenten rápidamente**.

Propuestas para la disminución de la autocorrelación:

- Si la simulación es **rápida**:
 - Guardar sólo **una de cada n** simulaciones generadas (parámetro *thin* = *n* de la simulación).
 - Este método **no cambia la convergencia** (simplemente disminuye el tamaño muestral guardado), pero disminuye los requerimientos de memoria al no guardar todas las iteraciones.
- Si la simulación es **lenta**:
 - Trataremos de **reparametrizar el modelo** (no es trivial ...), o cuestionarnos si el modelo que hemos propuesto está **bien planteado** o no.



- Las cadenas anteriores corresponden al siguiente **modelo (mal formulado, su posterior es impropia)**.

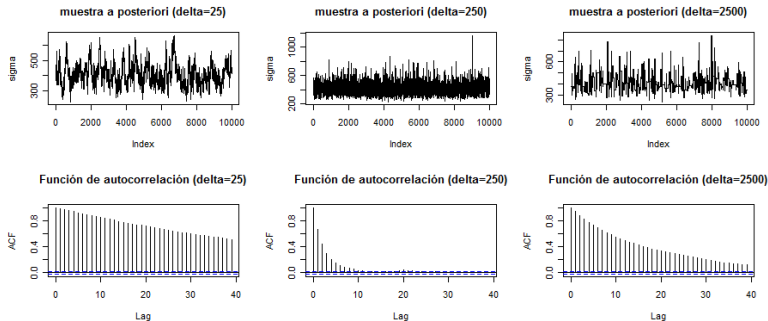
```
model{  
    x[1]~dnorm(x[2],1)  
    x[2]~dnorm(x[1],1)  
}
```

- La **mala convergencia sugiere la mala formulación** del modelo.

Sintonización de la función de propuesta en MH

- En los algoritmos **MH de caminata aleatoria** se utilizan funciones de propuesta (normal, uniforme, ...) centradas en el último valor de la cadena de Markov. En esos casos, la **elección del parámetro** de variabilidad de dichas funciones puede ser **importante**.
- Por ejemplo, para función de propuesta
$$q(\theta^{(*)} | \theta^{(i)}) = U(\theta^{(*)} | \theta^{(i)} - \delta, \theta^{(i)} + \delta)$$
 - Si δ es muy **pequeño**, $\theta^{(*)}$ será muy parecido a $\theta^{(i)}$ por tanto será **aceptado con alta probabilidad**. La cadena se mueve con **pasos muy pequeños**, exhibiendo alta autocorrelación.
 - Si δ es muy **grande**, $\theta^{(*)}$ puede situarse muy lejos de la distribución por lo que **raramente será aceptado**. La cadena de Markov **repite** muchos de sus **valores**, exhibiendo también alta autocorrelación.
- Por tanto, suelen resultar adecuados **valores** de δ **“moderados”**, pero ¿qué es moderado?

- Resulta óptimo valores de δ que conlleven una **probabilidad de aceptación** de cada paso del algoritmo MH en torno al **30 %-50 %**, así:
 - Si la probabilidad de aceptación es **más alta** $\rightarrow$ **incrementar** δ .
 - Si la probabilidad de aceptación es **más baja** $\rightarrow$ **disminuir** δ .



- En los casos anteriores las **probabilidades de aceptación** son del **93 %, 45 % y 5 %**, respectivamente.
- Los valores óptimos de δ se determinarán **preejecutando** el algoritmo con distintos valores de este parámetro.

Sección 2

Simulación MCMC con JAGS

- **MCMC** supone una **herramienta fundamental** para la estadística bayesiana, que hace posible la inferencia en un gran número de modelos que, antes de la aparición de MCMC, apenas resultaba posible.
- Sin embargo, el gran **inconveniente** de las técnicas MCMC, tal cual las conocéis hasta ahora es su gran **complejidad** desde un punto de vista práctico. Su implementación resulta muy laboriosa.
- Por suerte, disponemos de **lenguajes de programación**, implementados como paquetes de R, que facilitan esta labor.
- A principios de los 90 apareció un **lenguaje de modelización** que permitía implementar MCMC para prácticamente cualquier modelo: Bayesian inference Using Gibbs Sampling, **BUGS**.
- **WinBUGS**, el programa que implementó el lenguaje BUGS, **dejó de desarrollarse** en la década de los 2000.

- En 2003 surge **JAGS**, Just Another Gibbs Sampler, basado en BUGS, pero **más cómodo** para trabajar desde R.
- JAGS¹ es un programa **externo** a R que, una vez **formulado un modelo** estadístico, **construye** automáticamente un **algoritmo MCMC** para simular sus parámetros.
- R cuenta con **distintas librerías que llaman** directamente a JAGS de forma transparente.
- R **exportará** a JAGS el **modelo** bayesiano (en lenguaje BUGS) y los **datos** correspondientes. JAGS **devolverá** a R las **muestras** de las distribuciones a posteriori de los parámetros del modelo simuladas mediante MCMC.

¹Su ejecución requiere su instalación previa desde, por ejemplo:
<https://mcmc-jags.sourceforge.io/>

- En esta sección vamos a **describir** brevemente el funcionamiento del **lenguaje** BUGS que hay detrás de JAGS, que es el que utilizaremos.
- La **sintaxis** de cualquier modelo de JAGS deberá incluirse dentro de los corchetes de una **sentencia** `model{...}`.
- A diferencia de la sintaxis de R, la **sintaxis** de cualquier modelo JAGS **no** se interpreta de manera **secuencial** (cada línea detrás de la anterior) sino que el orden de las sentencias no importa. Por tanto, podemos definir por ejemplo las previas antes que la verosimilitud, o al revés.
- En un modelo de JAGS tanto **datos** como **parámetros** del modelo se entienden como **variables aleatorias**, unas observadas y otras no. JAGS muestreará, y por tanto hará **inferencia**, sobre toda aquella **variable** aleatoria que no **haya sido observada** (de la que no tenga datos).

- **Toda variable** aleatoria en JAGS deberá tener su **distribución**.
 - En el caso de los **datos** su distribución define la **verosimilitud** del modelo.
 - En el caso de **parámetros** del modelo su distribución define la correspondiente **distribución inicial**.
- La **sintaxis** para definir la distribución de cualquier variable aleatoria es:
`variable~distribucion(param1, param2, ...)`
 Por ejemplo²:

$$X \sim N(\mu, \sigma^2 = \tau^{-1}) \rightarrow X \sim \text{dnorm}(\text{mu}, \text{tau})$$
- Al igual que en R, las **distribuciones** en JAGS se definen por su **nombre precedido** de la **letra d**, por ejemplo: `dnorm`, `dbern`, `dbin`, `dpois`, `dunif`, `dgamma` ...³.

²La variabilidad de la distribución normal en JAGS se parametriza en función de la precisión(=1/varianza)

³Puedes encontrar una lista de las distribuciones disponibles en JAGS en la tabla 6.2 de su manual https://people.stat.sc.edu/hansont/stat740/jags_user_manual.pdf

- Se puede poner **comentarios** en modelos JAGS precedidos de una **almohadilla**

Ejemplo

- Los siguientes 2 **modelos** en lenguaje BUGS son **equivalentes**:

```
model{  
  # verosimilitud  
  for (i in 1:length(y))  
  {  
    y[i] ~ dbern(pi)  
  }  
  # a priori  
  pi ~ dunif(0,1)  
}
```

```
model{  
  # distribución inicial  
  pi ~ dbeta(1,1)  
  # verosimilitud  
  for (i in 1:length(y))  
  {  
    y[i] ~ dbern(pi)  
  }  
}
```

- Al correr ambos **modelos**, JAGS **simulará** valores de la distribución a posteriori de **pi**, la probabilidad de éxito para **y**.

- JAGS, además de variables aleatorias, permite el uso de **transformaciones deterministas** (funciones directas) de éstas, que son de gran utilidad.
- Su **sintaxis** es de la forma:

`nodo <- f(variables, constantes)`

donde `f` es la **transformación** oportuna.

- JAGS dispone de **funciones elementales** básicas (`exp`, `log`, `logit`, `sin`, `cos` ...) para la definición de las transformaciones anteriores⁴.
- Las funciones `log` y `logit` pueden utilizarse tanto a la **izquierda** como a la derecha del comando `<-` dado que muchas veces se utilizan como función de **enlace en GLMs**.

⁴Ver sección 6 del manual de JAGS para una descripción de las funciones disponibles.

Ejemplo

- El siguiente modelo utiliza el **operador** `<-` para parametrizar la normal en función de la **desviación típica** σ , en lugar de la **precisión**^a.
- También utilizamos el **operador** `<-` para obtener una muestra de σ^2 .

^aSuele ser muy habitual utilizar una distribución uniforme "vaga" como distribución inicial de la desviación típica en distribuciones normales

```
model{  
  # verosimilitud  
  for (i in 1:length(y))  
  {  
    y[i] ~ dnorm(mu,tau)  
  }  
  # a priori  
  mu ~ dnorm(0,0.0001)  
  tau <- sigma^(-2)  
  sigma ~ dunif(0,1000)  
  
  varianza <- 1/tau  
}
```

- JAGS **simulará** valores para μ , σ , τ y σ^2 . Las dos últimas **no** son **variables aleatorias genuinas** sino transformaciones de las anteriores.

- Tal y como hemos podido comprobar, JAGS admite el uso de **vectores** para almacenar datos, de la misma forma también admite el uso de **matrices y arrays**.
- La sintaxis de JAGS **no es tan vectorial** como la de R, normalmente hay que definir los elementos de vectores, matrices y arrays uno a uno con **sentencias for**. Sentencias del tipo:

$$y[] \sim N(\mu, \tau) \text{ o } x[] \leftarrow -y[] + z[]$$

para todo un vector en general son incorrectas.

- **Operadores** del tipo $<$, $>$, $\leq$, $\geq$, $==$, ... son habitualmente usados para **evaluar condiciones**, por ejemplo $P(\beta_0 > 0)$. Dichos operadores **devuelven TRUE o FALSE** en cada iteración del MCMC según se cumpla o no la condición.
- El **porcentaje de veces** que se cumpla la condición a evaluar (media del número de TRUE y FALSE devueltos por la condición) se usa para **estimar su probabilidad**.

Ejemplo

- Queremos estimar la **probabilidad** de que cierta variable categórica x valga **3 o 4**. En ese caso creamos la variable:

`P.x34 <- (x==3) + (x==4)`

La **media** de `P.x34` (frecuencia relativa de unos en la muestra) estimará la **probabilidad** buscada.

- Queremos estimar la **probabilidad** de que el parámetro β_0 de cierto modelo sea **mayor que 1**. En ese caso nos ayudaremos de la variable:

`P.beta0 <- beta0>1`

de la que tendremos que calcular **su media** para estimar dicha **probabilidad**.

- JAGS también puede muestrear de distribuciones **predictivas** ($P(x^* | \mathbf{x})$), simplemente tendremos que incluir en el modelo la **distribución** del valor **a muestrear** ($P(x^*|\theta)$). Al no tener valores observados para esa variable, JAGS **los simulará**.

Ejemplo

- El siguiente modelo además de muestrear de μ y σ , muestrea **valores** de la **predictiva** para un nuevo hipotético individuo *y.pred*.
- La **verosimilitud** permite **aprender** de los datos sobre μ y σ , que se utilizan para **predecir** *y.pred*.

```
model{  
  # verosimilitud  
  for (i in 1:length(y))  
  {  
    y[i] ~ dnorm(mu,tau)  
  }  
  # a priori  
  mu ~ dnorm(0,0.0001)  
  sigma ~ dunif(0,1000)  
  tau <- sigma^(-2)  
  #distribución predictiva  
  y.pred ~ dnorm(mu, tau)  
}
```


Ejecución de JAGS desde R

- Una vez conocemos el lenguaje BUGS nuestro **primer cometido** será **expresar** nuestro modelo en **ese lenguaje**.
- Una vez definido el correspondiente **modelo estadístico** en lenguaje BUGS, simularemos de él **llamándolo** desde R.
- Para ello haremos uso de la **librería** `jagsUI` de R, que deberemos instalar previamente⁵.
- La **función** `jags` de `jagsUI`:
 - **Exporta** a JAGS la información necesaria para correr el modelo.
 - **Corre** el modelo en JAGS.
 - **Importa** la información generada por JAGS y la **convierte** a un objeto de R con una estructura conveniente.

⁵junto a JAGS, disponible en <https://mcmc-jags.sourceforge.io/>

- La función `jags` tiene varios **argumentos obligatorios**:
 - `data`: **lista** (named list) **con los datos** necesarios para que JAGS corra el modelo. El **nombre** de cada elemento de la lista **deberá coincidir** con el nombre del correspondiente elemento del modelo.
 - `inits`: **lista con valores iniciales** de las variables de la cadena de Markov. El **nombre** de cada elemento de la lista deberá coincidir con el del correspondiente elemento del modelo. Suele ser **habitual** usar una **función generadora** de valores iniciales (ver siguiente ejemplo).
 - `parameters.to.save`: **vector** con los nombres (entre comillas) de las **variables** cuyos valores muestreados **se guardarán**.
 - `model.file`: **ruta del archivo** que contiene el modelo, habitualmente **exportado** con la función `cat` (ver siguiente ejemplo).
 - `n.chains`: **número de cadenas** a simular.
 - `n.iter`: número de **iteraciones** a simular **por cadena**.
- Además de los argumentos obligatorios, los siguientes argumentos opcionales también son de gran utilidad:
 - `n.burnin`: **número** de iteraciones de `n.iter` a descartar como **burn-in**.
 - `n.thin`: parámetro **thin** de la simulación. JAGS guardará **1 de cada `n.thin`** simulaciones.

- El número total de **simulaciones guardadas** por JAGS será:
`n.chains(n.iter-n.burnin)/n.thin`

Ejemplo (Colesterol (con JAGS))

- Analizamos los datos de **colesterol** que teníamos en el último ejemplo de la **sesión anterior**.
- **Escribimos el modelo** en lenguaje BUGS **y lo exportamos** al archivo texto `colesterol.jags`

El modelo asumía:

$$x_i \sim N(\mu, \tau), \quad i = 1, \dots, n$$

$$\mu \sim N(m, t)$$

$$\tau \sim \text{Gamma}(a, b)$$

```
cat(  
  "model{  
    # verosimilitud  
    for (i in 1:length(x))  
    {  
      x[i] ~ dnorm(mu,tau)  
    }  
    # a priori  
    mu ~ dnorm(m,t)  
    tau ~ dgamma(a,b)  
  }  
  ",file="colesterol.jags")
```

Ejemplo

- Definimos los **datos** a utilizar como una **lista**:

```
datos <- list(x = c(106, ...), m = 0, t = 1e-6, a = 1e-3, b =
```

- Generamos **valores iniciales** para las cadenas mediante una **función** que simule valores de la distribución oportuna:

```
valores.iniciales = function(){  
  list(mu = rnorm(1,100,20), tau=runif(1))}
```

jags llamará a la función **para cada cadena** inicializando las variables en todas ellas.

- Iteraciones, burn-in, thin y número de cadenas:

```
iteraciones = 10000; burnin = 1000; thin = 9; cadenas = 3
```

- Estamos en condiciones de **llamar ya a la función jags** de R:

```
resul.jags <- jagsUI::jags(model.file= "colesterol.jags",  
  data = datos, parameters.to.save = c("mu","tau"),  
  inits = valores.iniciales, n.chains= cadenas,  
  n.iter = iteraciones, n.burnin = burnin, n.thin = thin)
```

- `jags` devuelve un objeto de **clase** `jagsUI` que es, a su vez, una lista donde **destacan los siguientes elementos**:
 - `sims.list`: **lista**, con una componente para cada uno de los **elementos del vector** `parameters.to.save`. Cada uno de estas componentes contiene las **simulaciones** de la distribución a posteriori del correspondiente parámetro del modelo.
 - `summary`: resumen con **estadísticos descriptivos y de convergencia** (estadístico R de Gelman-Rubin ($R\text{-hat}$) y número de simulaciones efectivas (`n.eff`)) de cada una de las variables guardadas en la simulación.
- Al mostrar el resultado de `jags`, la **información mostrada** es, básicamente, la de la **componente** `summary` del objeto.
- La librería `MCMCvis` de R dispone de la función `MCMCtrace` de gran utilidad para **visualizar las cadenas** de Markov generadas para cada variable guardada.

Ejemplo (Colesterol (con JAGS))

- Vamos a **explorar** la **información** devuelta en la llamada anterior a `jags`.

Ejemplo

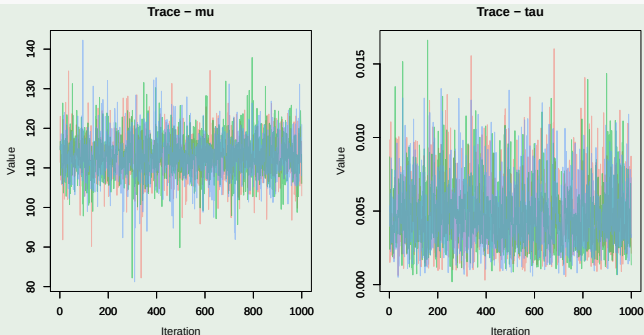
resul.jags

```
## JAGS output for model 'colesterol.jags', generated by jagsUI.
## Estimates based on 3 chains of 10000 iterations,
## adaptation = 100 iterations (sufficient),
## burn-in = 1000 iterations and thin rate = 9,
## yielding 3000 total samples from the joint posterior.
## MCMC ran for 0.002 minutes at time 2025-03-30 18:28:35.846945.
##
##           mean      sd    2.5%    50%   97.5% overlap0 f  Rhat n.eff
## mu       112.947 5.648 101.991 112.763 124.007   FALSE 1 1.000 3000
## tau        0.005 0.002   0.001   0.004   0.011   FALSE 1 1.001 3000
## deviance  75.073 2.273  72.859  74.437  81.075   FALSE 1 1.000 3000
##
## Successful convergence based on Rhat values (all < 1.1).
## Rhat is the potential scale reduction factor (at convergence, Rhat=1).
## For each parameter, n.eff is a crude measure of effective sample size.
##
## overlap0 checks if 0 falls in the parameter's 95% credible interval.
## f is the proportion of the posterior with the same sign as the mean;
## i.e., our confidence that the parameter is positive or negative.
##
## DIC info: (pD = var(deviance)/2)
## pD = 2.6 and DIC = 77.657
## DIC is an estimate of expected predictive error (lower is better).
```

Ejemplo

- También podemos **visualizar las cadenas** generadas en la simulación:

```
MCMCvis::MCMCtrace(resul.jags,params = c("mu","tau"),  
                    pdf=FALSE, type="trace")
```



- Cada color** corresponde a cada una de las cadenas.
- La **convergencia** parece **buen**a, tal y como anticipaban los estadísticos $\hat{R}$ y n_{eff} .